SQL
INTERVIEW MASTER GUIDE

Every question, query and output - explained in plain language with memory hooks and ready-to-speak answers

Prepared for: jashu5446
5 October 2026

INDEX

SQL INTERVIEW MASTER GUIDE 6

SECTION 1 — DATABASE & SQL FUNDAMENTALS (the warm-up questions) 7

Q: What is a database, DBMS, RDBMS, and SQL? (they ask all four, often together) 7

Q: What are the types of SQL commands? (DDL, DML, DQL, DCL, TCL — guaranteed question) 7

Q: What is a primary key, foreign key, unique key, and NOT NULL? (the constraints family) 8

Q: What are the common data types? (quick table — they test recognition) 8

Q: What is NULL — and why does = NULL never work? (a favourite trick question) 8

SECTION 2 — HOW A QUERY ACTUALLY RUNS (the flow — asked as "what is the order of execution?") 10

Q: What is the order of execution? (the short answer first) 10

Q: GROUP BY — Syntax, Table, Query, Output 10

Q: HAVING — Syntax, Table, Query, Output 11

Q: ORDER BY — Syntax, Table, Query, Output 11

Q: LIMIT and OFFSET — Syntax, Table, Query, Output 12

Q: DISTINCT — Syntax, Table, Query, Output 13

Q: All of them in ONE query (the order they run, top to bottom) 13

SECTION 3 — DDL & TABLE MANAGEMENT 14

Q: CREATE TABLE — the full syntax with every constraint 14

Q: ALTER TABLE — add, modify, drop 14

Q: TRUNCATE vs DELETE vs DROP (a dedicated comparison — see also Section 10) 14

Q: INSERT, UPDATE, DELETE — with their outputs 15

SECTION 4 — SELECT & FILTERING (the daily bread) 16

Q: The SELECT statement — every clause in order 16

Q: Which operators does WHERE support? (comparison, logical, pattern) 16

Q: ORDER BY — single, multiple columns, and directions 16

Q: LIMIT and OFFSET — pagination 17

Q: SELECT DISTINCT — removing duplicates 17

SECTION 5 — JOINS (every join written out: Syntax, Query, Output) 18

The demo data (every join below uses these tables) 18

Q: INNER JOIN — only the rows that match on BOTH sides 18

Q: LEFT JOIN — every row from the LEFT table, matched or not 19

Q: RIGHT JOIN — every row from the RIGHT table, matched or not 19

Q: FULL OUTER JOIN — every row from BOTH tables 20

Q: CROSS JOIN — every possible combination (the cartesian product) . . . 21
Q: SELF JOIN — a table joined to itself (employee and manager) . . . 21
Q: Joining THREE tables — the pattern . . . 22
The joins in one line each (the memory sheet) . . . 22

SECTION 6 — AGGREGATIONS: GROUP BY, HAVING, AND THE AGGREGATE FUNCTIONS . . . 23

Q: What do the aggregate functions do? . . . 23
Q: GROUP BY — collapsing rows into groups . . . 23
Q: HAVING — filtering the GROUPS . . . 23
Q: WHERE + GROUP BY + HAVING in one query — the canonical shape . . . 24
Q: Counting with zero — "show all departments including empty ones" (the LEFT JOIN + COUNT question) . . . 24

SECTION 7 — SUBQUERIES . . . 25

Q: What is a subquery, and where can it sit? . . . 25
Q: Scalar subquery — a single value . . . 25
Q: Multi-row subquery — IN, ANY, ALL . . . 25
Q: Correlated subquery — the inner query references the outer row . . . 25
Q: EXISTS vs IN — the performance question . . . 26

SECTION 8 — WINDOW FUNCTIONS (explained from zero, step by step) . . . 27

Q: What is a window function? (start with this one idea) . . . 27
Q: The syntax — OVER, PARTITION BY, ORDER BY (the three words) . . . 28
Q: ROW_NUMBER, RANK, DENSE_RANK — numbering the rows . . . 28
Q: LAG and LEAD — look at the previous or next row . . . 30
Q: Running total — SUM with ORDER BY (and no PARTITION) . . . 30
Q: Top 2 earners per department (the real-world use — window inside a subquery) . . . 31
Q: The two rules beginners always trip on . . . 31

SECTION 9 — CTEs (WITH clauses) . . . 33

Q: What is a CTE, and why use it instead of a subquery? . . . 33
Q: Multiple CTEs in one query . . . 33
Q: Recursive CTE — the org hierarchy (the "show the chain of command" question) . . . 33

SECTION 10 — THE CLASSIC DIFFERENCES (the section they open every SQL interview with) . . . 35

Q: WHERE vs HAVING . . . 35
Q: GROUP BY vs ORDER BY . . . 35
Q: DELETE vs TRUNCATE vs DROP . . . 35
Q: UNION vs UNION ALL . . . 35
Q: IN vs EXISTS . . . 36
Q: Primary key vs Unique key . . . 36
Q: Clustered vs Non-clustered index . . . 36

Q: CHAR vs VARCHAR . . . 36
Q: COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col) . . . 37
Q: RANK vs DENSE_RANK vs ROW_NUMBER . . . 37
Q: DDL vs DML . . . 37
Q: Normalization vs Denormalization . . . 37
Q: View vs Table . . . 37
Q: Stored Procedure vs Function . . . 37
Q: TRUNCATE and rollback — the gotcha . . . 38
Q: AND vs OR precedence . . . 38
Q: NULL vs empty string . . . 38
Q: HAVING without GROUP BY . . . 38

SECTION 11 — TRANSACTIONS & CONCURRENCY . . . 39

Q: What is a transaction, and what is ACID? . . . 39
Q: The TCL commands . . . 39
Q: What are the isolation levels? (the levels table) . . . 39
Q: What is a deadlock? . . . 39

SECTION 12 — RDBMS THEORY QUESTIONS . . . 40

Q: What is normalization? Explain 1NF, 2NF, 3NF with a tiny example. . . . 40
Q: What is denormalization and when would you use it? . . . 40
Q: What is an entity-relationship model, and what are the relationship types? . . . 40
Q: What is referential integrity? . . . 40
Q: What is a schema? What is an instance? . . . 40
Q: SQL vs NoSQL — when do you choose which? . . . 40
Q: What are the common integrity constraints? . . . 41
Q: What is a view, a stored procedure, a trigger, a cursor? . . . 41
Q: What is the difference between a database and a data warehouse? . . . 41

SECTION 13 — THE QUERY PROBLEM BANK (the ones they make you write on the spot) . . . 42

Q13.1 Second highest salary (the single most-asked query — know all three ways) . . . 42
Q13.2 Nth highest salary (make it a parameter) . . . 42
Q13.3 Highest salary PER DEPARTMENT (the GROUP BY + join classic) . . . 42
Q13.4 The employee(s) earning the top salary in EACH department (needs a window, not just MAX) . . . 43
Q13.5 Find duplicate rows (and delete them keeping one) . . . 43
Q13.6 Employees earning more than their manager (self join — a favourite) . . . 43
Q13.7 Departments with NO employees (the anti-join) . . . 43
Q13.8 Employees hired in the last 30 days (date filtering) . . . 44
Q13.9 Running total of salaries in hire order (window) . . . 44
Q13.10 Employees earning above the company average . . . 44

Q13.11 Employees earning above THEIR department's average (correlated) . . . 44
Q13.12 Count employees per department INCLUDING zero (the COUNT trap) . . . 44
Q13.13 Second-highest salary per department (window, per group) . . . 44
Q13.14 Find the manager's name for every employee (self join, LEFT to keep bosses) . . . 45
Q13.15 Rows where two columns have the same value (the "same value" question) . . . 45
Q13.16 Salary bands with CASE (conditional column — very common) . . . 45
Q13.17 Count of employees per manager (including managers with none) . . . 45
Q13.18 Employees with no manager (top of the chain) . . . 45
Q13.19 The highest-paid employee overall (name, not just number) . . . 45
Q13.20 Percentage share of each department in total salary . . . 45
Q13.21 Pagination — page 2 of 3-per-page . . . 46
Q13.22 Swap two values in a column (the tricky update) . . . 46
Q13.23 Employees whose name starts with a vowel . . . 46
Q13.24 Find missing numbers in a sequence (the "gap" question) . . . 46
Q13.25 Top 2 salaries per department (top-N per group — window) . . . 46

SECTION 14 — DATABASE TESTING FOR QA (because you are a tester, not a DBA) . . . 47

Q: As a QA engineer, what do you actually test in a database? . . . 47
Q: How do you validate an ETL / data migration? . . . 47
Q: The common DB-testing interview questions, one line each . . . 47

SECTION 15 — RAPID-FIRE, FULLY EXPLAINED (final revision) . . . 48

SECTION 16 — CLOSING ANSWER + REVISION PLAN . . . 49

Last-day revision plan . . . 49

SQL INTERVIEW MASTER GUIDE

Every question they ask, explained in plain language — with tables, queries, their OUTPUT, memory hooks, and ready-to-speak interview answers.

How to use this guide: read a topic, then close the page and reproduce the query from memory. Every example runs on ONE small dataset (shown in Section 5) so you can trace every result by hand. Syntax is MySQL-flavoured (the most common in Indian QA interviews); where Oracle/PostgreSQL differ, the difference is called out inline.

SECTION 1 — DATABASE & SQL FUNDAMENTALS (the warm-up questions)

Q: What is a database, DBMS, RDBMS, and SQL? (they ask all four, often together)

Understand it: A **database** is organised storage for data — like a well-run filing cabinet, not a pile of papers. A **DBMS** is the software that manages that cabinet (store, retrieve, secure, back up) — e.g. MySQL, Oracle, MongoDB. An **RDBMS** is a DBMS that stores data as RELATED tables (rows and columns with keys linking them) — MySQL, Oracle, PostgreSQL, SQL Server. **SQL** is the LANGUAGE you speak to an RDBMS — it is not a database itself; it is the grammar.

The one-line distinctions to recite:

Term	One line
Database	organised collection of data
DBMS	software that manages databases (MySQL, Oracle)
RDBMS	DBMS with tables + relationships + keys (the SQL world)
SQL	the query LANGUAGE for RDBMS
MySQL / Oracle	a specific RDBMS product that speaks SQL

MEMORY HOOK: *"Database = the cabinet, DBMS = the manager, RDBMS = related tables, SQL = the language. Cabinet, manager, relationships, grammar."*

SAY THIS: "A database is organised storage; a DBMS is the software managing it; an RDBMS is a DBMS that keeps data in related tables linked by keys; and SQL is the language I use to talk to it. So MySQL is an RDBMS, and SQL is what I write inside it."

Q: What are the types of SQL commands? (DDL, DML, DQL, DCL, TCL — guaranteed question)

Understand it: SQL commands are grouped by WHAT THEY DO to the database. Interviewers ask for the five families and one example command from each — get the family and the verb right and this question is free marks.

Family	Full form	What it does	Commands
DDL	Data Definition Language	defines/alters the STRUCTURE	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`
DML	Data Manipulation Language	changes the DATA inside tables	`INSERT`, `UPDATE`, `DELETE`
DQL	Data Query Language	READS data	`SELECT`
DCL	Data Control Language	controls PERMISSIONS	`GRANT`, `REVOKE`
TCL	Transaction Control Language	controls TRANSACTIONS	`COMMIT`, `ROLLBACK`, `SAVEPOINT`

The trap inside the trap: `TRUNCATE` is DDL (it changes structure-level state and cannot be rolled back in most engines), while `DELETE` is DML (row-by-row, transactional, rollback-able). Interviewers love that distinction.

MEMORY HOOK: *"Define, Manipulate, Query, Control, Transact — DDL, DML, DQL, DCL, TCL."*

SAY THIS: "SQL commands come in five families: DDL defines structure — CREATE, ALTER, DROP, TRUNCATE; DML changes data — INSERT, UPDATE, DELETE; DQL reads — SELECT; DCL controls permissions — GRANT, REVOKE; and TCL controls transactions — COMMIT, ROLLBACK, SAVEPOINT. The one people trip on is TRUNCATE: it looks like a delete but it's DDL."

Q: What is a primary key, foreign key, unique key, and NOT NULL? (the constraints family)

Understand it: Constraints are RULES the database enforces so bad data can never enter. Each constraint answers one question.

Constraint	The rule it enforces	Key facts
PRIMARY KEY	uniquely identifies each row	unique + NOT NULL, only ONE per table
FOREIGN KEY	value must exist in another table	creates the parent-child LINK; blocks orphan rows
UNIQUE	no duplicates in this column	allows ONE NULL; can have many per table
NOT NULL	value must always be present	the column can never be empty
CHECK	value must satisfy a condition	e.g. `salary > 0`
DEFAULT	auto-value when none supplied	e.g. `status DEFAULT 'active'`

```
CREATE TABLE employees (
    emp_id     INT PRIMARY KEY,                 -- unique + not null, one per table
    emp_name   VARCHAR(50) NOT NULL,
    dept_id    INT,
    salary     DECIMAL(10,2) CHECK (salary > 0),
    status     VARCHAR(10) DEFAULT 'active',
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)   -- the link
);
```

The classic follow-up — PK vs UNIQUE: both stop duplicates, but a primary key is NOT NULL by definition and there is only ONE; a unique column may hold NULL and you can have several unique columns.

MEMORY HOOK: *"Primary = the ID card (one per person, never blank). Foreign = the reference letter (must point to a real person). Unique = no twins allowed. Check = the bouncer."*

SAY THIS: "A primary key uniquely identifies a row — unique and never null, one per table. A foreign key enforces a link: its value must exist in the referenced table, which is what stops orphan rows. A unique key also blocks duplicates but allows one null and you can have many. And CHECK is a value rule like salary greater than zero."

Q: What are the common data types? (quick table — they test recognition)

Type	Stores	Use for
`INT` / `BIGINT`	whole numbers	ids, counts
`DECIMAL(p,s)`	exact decimals	money, salary (never FLOAT for money!)
`FLOAT` / `DOUBLE`	approximate decimals	scientific values only
`CHAR(n)`	fixed-length text	codes that are always the same length
`VARCHAR(n)`	variable-length text	names, emails
`DATE` / `TIME` / `DATETIME`	date/time values	hire_date, created_at
`BOOLEAN`	true/false	flags
`TEXT` / `BLOB`	large text / binary	descriptions, files

The money rule to say out loud: "I use DECIMAL for money, not FLOAT — FLOAT stores approximations, and rounding a salary by a paisa is a bug."

Q: What is NULL — and why does `= NULL` never work? (a favourite trick question)

Understand it: NULL is not zero and not empty text — it means **"unknown"**. And you cannot compare against an unknown: `salary = NULL` evaluates to *unknown*, never true, so it silently returns no rows. That is why SQL has `IS NULL` / `IS NOT NULL` — special operators for the unknown.

```
-- WRONG — returns zero rows, silently:
SELECT * FROM employees WHERE dept_id = NULL;

-- RIGHT:
SELECT * FROM employees WHERE dept_id IS NULL;
```

The three NULL rules that win the follow-ups:

1. **Comparisons with NULL are unknown** — `NULL = NULL` is NOT true; `NULL <> NULL` is also not true.
2. **Aggregates skip NULLs** — `COUNT(col)` ignores nulls, `AVG(col)` averages only non-null values. `COUNT(*)` counts every row.
3. **NULL in arithmetic poisons the result** — `salary + NULL` is NULL, so a null column can silently blank out your calculated column (use `COALESCE(col, 0)`).

```
SELECT COUNT(*)          AS all_rows,       -- 8 — every row
       COUNT(dept_id)    AS with_dept,      -- 7 — skips the null dept
       AVG(salary)       AS avg_salary
FROM employees;

SELECT emp_name, COALESCE(dept_id, 0) AS dept_or_zero   -- replace the unknown
FROM employees;
```

MEMORY HOOK: *"NULL = unknown. You can't equal an unknown — use IS NULL. COUNT(*)* counts rows; COUNT(col) counts knowns."*

SAY THIS: "NULL means unknown, not zero or empty. Any comparison with NULL is unknown, so 'equals NULL' returns nothing — that's why IS NULL exists. Two more rules I keep straight: aggregates ignore NULLs, so COUNT(column) skips them while COUNT(*) counts every row; and NULL poisons arithmetic, so I wrap nullable columns in COALESCE when calculating."

SECTION 2 — HOW A QUERY ACTUALLY RUNS (the flow — asked as "what is the order of execution?")

Q: What is the order of execution? (the short answer first)

You WRITE a query in one order; SQL RUNS it in a different order. That one fact explains most "why doesn't my query work" questions.

Runs	Clause	What it operates on
1	**FROM / JOIN**	builds the working set of rows
2	**WHERE**	filters individual ROWS
3	**GROUP BY**	collapses rows into GROUPS
4	**HAVING**	filters the GROUPS
5	**SELECT**	computes the output columns
6	**DISTINCT**	removes duplicate result rows
7	**ORDER BY**	sorts the result
8	**LIMIT / OFFSET**	returns the requested slice

Three errors this order explains:

1. You cannot use a SELECT alias in WHERE — at WHERE time the alias does not exist yet.
2. You cannot use an aggregate in WHERE — `WHERE COUNT(*) > 1` is invalid; that is HAVING's job.
3. WHERE removes rows BEFORE grouping, so it changes what the groups even contain.

MEMORY HOOK: *"From-Where-Group-Having-Select-Distinct-Order-Limit — say it like a phone number."*

Every clause below is shown in the SAME four parts: Syntax, Table, Query, Output — so the flow is easy to compare.

Q: GROUP BY — Syntax, Table, Query, Output

Syntax: `SELECT column, aggregate() FROM table GROUP BY column;`

Table (input) — employees:

emp_id	emp_name	dept_id	salary
1	Asha	1	90000
2	Ravi	1	70000
3	Meera	1	70000
4	Kiran	2	60000
5	Divya	2	80000
6	Arjun	2	50000
7	Nisha	3	45000
8	Vikram	NULL	55000

Query:

```
SELECT dept_id, COUNT(*) AS headcount, AVG(salary) AS avg_salary
FROM   employees
GROUP BY dept_id;
```

Output — one row PER GROUP (8 rows in, 4 groups out):

dept_id	headcount	avg_salary
1	3	76666.67
2	3	63333.33
3	1	45000.00
NULL	1	55000.00

What happened: the 8 rows were bucketed by `dept_id`, and each aggregate ran PER BUCKET. The NULL dept_ids form their own group.

What `COUNT(*)` actually counts (the part people miss): `COUNT(*)` counts the ROWS in each group — it is NOT counting the `dept_id` column. The GROUP BY column only decides WHICH BUCKET a row falls into; the aggregate then runs over each bucket's rows. So `COUNT(*)` gives 3 for dept 1 because three rows landed in that bucket. If you wrote `COUNT(dept_id)` instead, it would count only the non-NULL `dept_id` values per group — so the NULL group would show 0, not 1. (The `*` means "any row", so NULLs still count.)

The rule: every selected column must be inside an aggregate OR listed in GROUP BY.

When it runs: step 3 — after WHERE, before HAVING.

Q: HAVING — Syntax, Table, Query, Output

Syntax: `SELECT column, aggregate() FROM table GROUP BY column HAVING aggregate_condition;`

Table (input) — employees: same table as above.

Query:

```
SELECT dept_id, COUNT(*) AS headcount
FROM   employees
GROUP BY dept_id
HAVING COUNT(*) > 1;
```

Output — only the GROUPS that pass (4 groups in, 2 out):

dept_id	headcount
1	3
2	3

What happened: GROUP BY made 4 groups (1, 2, 3, NULL); HAVING kept only those with more than one member — dept 3 and the NULL group were dropped.

Why it can't be WHERE: `WHERE COUNT(*) > 1` is invalid — WHERE runs before the groups exist.

When it runs: step 4 — right after GROUP BY.

Q: ORDER BY — Syntax, Table, Query, Output

Syntax: `SELECT columns FROM table ORDER BY column [ASC | DESC];`

Table (input) — employees: same table as above.

Query:

```
SELECT emp_name, salary
FROM   employees
ORDER BY salary DESC;
```

Output — same 8 rows, just sorted (8 in, 8 out):

emp_name	salary
Asha	90000
Divya	80000
Ravi	70000
Meera	70000
Kiran	60000
Vikram	55000
Arjun	50000
Nisha	45000

What happened: the row count did NOT change — ORDER BY only rearranges rows. (ASC is ascending and is the default; DESC is descending.)

Multi-column sort: `ORDER BY dept_id ASC, salary DESC` sorts by department first, then by salary high-to-low inside each department.

When it runs: step 7 — after SELECT and DISTINCT, near the very end.

Q: LIMIT and OFFSET — Syntax, Table, Query, Output

Syntax: `SELECT columns FROM table ORDER BY column LIMIT n OFFSET m;` — LIMIT = how many rows, OFFSET = how many to skip first.

Table (input) — employees: same table as above.

Query 1 — the top 3 (page 1):

```
SELECT emp_name, salary
FROM   employees
ORDER BY salary DESC
LIMIT 3;
```

Output (first 3 rows of the sorted result):

emp_name	salary
Asha	90000
Divya	80000
Ravi	70000

Query 2 — the NEXT 3 (page 2), using OFFSET:

```
SELECT emp_name, salary
FROM   employees
ORDER BY salary DESC
LIMIT 3 OFFSET 3;
```

Output (skip 3, then take 3):

emp_name	salary
Meera	70000
Kiran	60000
Vikram	55000

What happened: LIMIT cuts the sorted result to n rows; OFFSET skips m rows before cutting. Page 2 = `OFFSET (page-1) x pageSize` = 3.

The rule: LIMIT without ORDER BY returns an ARBITRARY slice — always sort first, then cut.

When it runs: step 8 — the very last step.

Q: DISTINCT — Syntax, Table, Query, Output

Syntax: `SELECT DISTINCT column FROM table;`

Table (input) — employees: same table as above.

Query:

```
SELECT DISTINCT dept_id FROM employees;
```

Output — 8 rows in, 4 out:

dept_id
1
2
3
NULL

What happened: every repeated `dept_id` collapsed to one — and all NULLs count as a single value, even though `NULL = NULL` is never true in a comparison.

DISTINCT vs GROUP BY: DISTINCT only removes duplicates; GROUP BY collapses rows so you can AGGREGATE them (`COUNT(*)` beside each group is only possible with GROUP BY).

When it runs: step 6 — after SELECT, before ORDER BY.

Q: All of them in ONE query (the order they run, top to bottom)

Query:

```
SELECT DISTINCT dept_id, COUNT(*) AS headcount
FROM   employees                -- step 1: the table
WHERE  salary > 45000           -- step 2: filter ROWS
GROUP BY dept_id                -- step 3: make GROUPS
HAVING COUNT(*) > 1             -- step 4: filter GROUPS
ORDER BY headcount DESC         -- step 7: sort the result
LIMIT 2;                        -- step 8: take the top 2
```

Output:

dept_id	headcount
1	3
2	3

Why this order and no other: WHERE must run before GROUP BY (you filter rows before you group them); HAVING must run after GROUP BY (there are no groups to filter before); SELECT runs after both because the aggregate `COUNT(*)` needs the groups; ORDER BY and LIMIT run last because they act on the finished result. That is the whole flow — each clause can only use what the clauses before it have produced.

MEMORY HOOK: *"Filter rows (WHERE), then group (GROUP BY), then filter groups (HAVING), then pick columns (SELECT), then sort (ORDER BY), then cut (LIMIT)."*

SAY THIS: "SQL runs in a fixed order: FROM and JOIN first, then WHERE on individual rows, then GROUP BY, then HAVING on the groups, then SELECT computes the columns, then DISTINCT, ORDER BY and LIMIT. So WHERE can only see rows — that's why aggregates go in HAVING — and an alias defined in SELECT isn't visible to WHERE because it hasn't been computed yet."

SECTION 3 — DDL & TABLE MANAGEMENT

Q: CREATE TABLE — the full syntax with every constraint

```
CREATE TABLE departments (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(50) NOT NULL UNIQUE,
    location   VARCHAR(50)
);

CREATE TABLE employees (
    emp_id     INT PRIMARY KEY AUTO_INCREMENT,
    emp_name   VARCHAR(50) NOT NULL,
    dept_id    INT,
    salary     DECIMAL(10,2) CHECK (salary > 0),
    hire_date  DATE,
    manager_id INT,
    FOREIGN KEY (dept_id)    REFERENCES departments(dept_id),
    FOREIGN KEY (manager_id) REFERENCES employees(emp_id)     -- self-reference!
);
```

The detail interviewers probe: `manager_id` referencing `employees` itself is a SELF-REFERENCING foreign key — the standard way to model a hierarchy (every employee points at their manager). Expect a follow-up query on it in Section 13.

Q: ALTER TABLE — add, modify, drop

```
ALTER TABLE employees ADD COLUMN email VARCHAR(100);        -- add a column
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2);   -- change type/size
ALTER TABLE employees DROP COLUMN email;                    -- remove a column
ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);  -- add a constraint
```

Q: TRUNCATE vs DELETE vs DROP (a dedicated comparison — see also Section 10)

```
DELETE FROM employees WHERE dept_id = 3;   -- DML: removes ROWS, rollback-able
TRUNCATE TABLE employees;                  -- DDL: empties the TABLE, resets identity
DROP TABLE employees;                      -- DDL: removes the TABLE itself
```

	DELETE	TRUNCATE	DROP
Removes	selected rows	ALL rows	the whole table + structure
Can filter (WHERE)?	yes	no	n/a
Type	DML	DDL	DDL
Rollback	yes	usually no	no
Speed	slow (row-by-row, logged)	fast	instant
Table after	still exists	exists but empty	GONE

MEMORY HOOK: *"Delete takes rows, truncate empties the table, drop deletes the table."*

Q: INSERT, UPDATE, DELETE — with their outputs

```
INSERT INTO departments (dept_id, dept_name, location)
VALUES (1, 'Engineering', 'Bangalore'),
       (2, 'Sales', 'Mumbai'),
       (3, 'HR', 'Delhi'),
       (4, 'Legal', 'Pune');          -- multi-row insert: 4 rows inserted

UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 2;                    -- raise Sales by 10%

DELETE FROM employees
WHERE emp_name = 'Arjun';             -- one row gone
```

The safety habit to mention: "For any UPDATE or DELETE I first run the same WHERE with a SELECT to see exactly which rows will be hit — a missing WHERE clause touches every row."

```
SELECT * FROM employees WHERE dept_id = 2;   -- check FIRST, then update
```

SECTION 4 — SELECT & FILTERING (the daily bread)

Q: The SELECT statement — every clause in order

```
SELECT DISTINCT dept_id, AVG(salary) AS avg_sal
FROM   employees
WHERE  hire_date > '2020-01-01'
GROUP BY dept_id
HAVING AVG(salary) > 40000
ORDER BY avg_sal DESC
LIMIT  3;
```

The clause order to memorise: `SELECT -> FROM -> WHERE -> GROUP BY -> HAVING -> ORDER BY -> LIMIT`.

Q: Which operators does WHERE support? (comparison, logical, pattern)

```
-- comparison
WHERE salary >= 50000
WHERE dept_id <> 2                -- <> and != are the same "not equal"

-- logical (AND / OR / NOT — AND binds tighter than OR!)
WHERE dept_id = 1 AND salary > 60000
WHERE dept_id = 1 OR  dept_id = 2
WHERE NOT (dept_id = 3)

-- BETWEEN (inclusive on both ends!)
WHERE salary BETWEEN 50000 AND 80000

-- IN (a list, cleaner than many ORs)
WHERE dept_id IN (1, 2)

-- LIKE (pattern match: % = any number of chars, _ = exactly one)
WHERE emp_name LIKE 'A%'          -- starts with A
WHERE emp_name LIKE '%a'          -- ends with a
WHERE emp_name LIKE '_avi'        -- exactly one char, then 'avi' -> Ravi
WHERE emp_name LIKE '%vi%'        -- contains 'vi'
```

The precedence trap to recite: `WHERE dept_id = 1 OR dept_id = 2 AND salary > 60000` does NOT mean what it reads — AND runs first, so it becomes "dept 1 (any salary) OR (dept 2 with salary > 60000)". Always bracket your intent.

> **MEMORY HOOK:** *"BETWEEN includes the edges, IN is a clean OR-list, % is many characters and _ is exactly one."*

Q: ORDER BY — single, multiple columns, and directions

```
SELECT emp_name, dept_id, salary
FROM   employees
ORDER BY salary DESC;                -- highest first

SELECT emp_name, dept_id, salary
FROM   employees
ORDER BY dept_id ASC, salary DESC;    -- dept ascending, then salary descending inside it
```

Result of the second query (our dataset, dept ordered, salary high-to-low within each dept):

emp_name	dept_id	salary
Asha	1	90000
Ravi	1	70000
Meera	1	70000
Divya	2	80000
Kiran	2	60000
Arjun	2	50000
Nisha	3	45000

The NULL rule: `ORDER BY` puts NULLs together at one end (MySQL shows them first in ASC). Use `ORDER BY col IS NULL, col` to force NULLs last when it matters.

Q: LIMIT and OFFSET — pagination

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 3;          -- top 3
SELECT * FROM employees ORDER BY salary DESC LIMIT 3 OFFSET 3; -- rows 4-6 (page 2)
```

The rule that makes pagination correct: `LIMIT` without `ORDER BY` returns an ARBITRARY slice — always sort first, then cut. That single sentence answers half the pagination questions.

Q: SELECT DISTINCT — removing duplicates

```
SELECT DISTINCT dept_id FROM employees;
```

dept_id
1
2
3
NULL

The NULL quirk: DISTINCT treats all NULLs as one — even though `NULL = NULL` is never true for comparisons.

SECTION 5 — JOINS (every join written out: Syntax, Query, Output)

The demo data (every join below uses these tables)

employees (8 rows)

emp_id	emp_name	dept_id	salary	manager_id
1	Asha	1	90000	NULL
2	Ravi	1	70000	1
3	Meera	1	70000	1
4	Kiran	2	60000	NULL
5	Divya	2	80000	4
6	Arjun	2	50000	4
7	Nisha	3	45000	NULL
8	Vikram	NULL	55000	NULL

departments (4 rows)

dept_id	dept_name	location_id
1	Engineering	1
2	Sales	2
3	HR	3
4	Legal	4

locations (4 rows) — used only in the three-table join at the end

location_id	city
1	Bangalore
2	Mumbai
3	Delhi
4	Pune

Two rows to watch the whole way down: Legal (dept 4) has **no employees**, and Vikram has **no department**. Every join behaves differently because of these two rows.

Q: INNER JOIN — only the rows that match on BOTH sides

Syntax:

```
SELECT columns
FROM   table1
INNER JOIN table2 ON table1.column = table2.column;
```

Query:

```
SELECT e.emp_name, d.dept_name
FROM   employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Output — 7 rows. Vikram (no department) is dropped, and Legal (no employees) never appears:

emp_name	dept_name
Asha	Engineering
Ravi	Engineering
Meera	Engineering
Kiran	Sales
Divya	Sales
Arjun	Sales
Nisha	HR

Q: LEFT JOIN — every row from the LEFT table, matched or not

Syntax:

```
SELECT columns
FROM   table1
LEFT JOIN table2 ON table1.column = table2.column;
```

Query:

```
SELECT e.emp_name, d.dept_name
FROM   employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

Output — 8 rows. Every employee is here; Vikram has no department, so dept_name is NULL:

emp_name	dept_name
Asha	Engineering
Ravi	Engineering
Meera	Engineering
Kiran	Sales
Divya	Sales
Arjun	Sales
Nisha	HR
Vikram	NULL

Q: RIGHT JOIN — every row from the RIGHT table, matched or not

Syntax:

```
SELECT columns
FROM   table1
RIGHT JOIN table2 ON table1.column = table2.column;
```

Query:

```
SELECT e.emp_name, d.dept_name
FROM   employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

Output — 8 rows. Every department is here; Legal has no employees, so emp_name is NULL:

emp_name	dept_name
Asha	Engineering
Ravi	Engineering
Meera	Engineering
Kiran	Sales
Divya	Sales
Arjun	Sales
Nisha	HR
NULL	Legal

Q: FULL OUTER JOIN — every row from BOTH tables

What it is, in plain words: FULL JOIN returns ALL rows — the ones that match, PLUS the left rows that have no match (right side filled with NULL), PLUS the right rows that have no match (left side filled with NULL). It is LEFT JOIN and RIGHT JOIN combined. The word OUTER is optional — FULL JOIN and FULL OUTER JOIN mean exactly the same thing.

Syntax:

```
SELECT columns
FROM   table1
FULL OUTER JOIN table2 ON table1.column = table2.column;
```

MySQL has no FULL OUTER JOIN keyword — you write it as a LEFT JOIN UNION a RIGHT JOIN:

Query:

```
SELECT e.emp_name, d.dept_name
FROM   employees e LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name
FROM   employees e RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

Output — 9 rows. Both "orphans" now appear: Vikram with a NULL department, AND Legal with a NULL employee:

emp_name	dept_name
Asha	Engineering
Ravi	Engineering
Meera	Engineering
Kiran	Sales
Divya	Sales
Arjun	Sales
Nisha	HR
Vikram	NULL
NULL	Legal

Why 9 and not 16: LEFT gave 8 rows and RIGHT gave 8 rows, but 7 rows are the same in both — UNION removes those duplicates, leaving 7 matched + Vikram + Legal = 9.

Q: CROSS JOIN — every possible combination (the cartesian product)

What it is, in plain words: there is NO match rule at all. Every row of the first table is paired with every row of the second — 8 employees x 4 departments = 32 rows. It is rarely what you want directly; its real importance is that a JOIN with a FORGOTTEN ON clause behaves exactly like this and floods your result.

Syntax:

```
SELECT columns
FROM   table1
CROSS JOIN table2;
```

Query:

```
SELECT e.emp_name, d.dept_name
FROM   employees e
CROSS JOIN departments d;
```

Output — 32 rows (8 x 4). The first 8 are shown; every employee then repeats for all 4 departments:

emp_name	dept_name
Asha	Engineering
Asha	Sales
Asha	HR
Asha	Legal
Ravi	Engineering
Ravi	Sales
Ravi	HR
Ravi	Legal
...	(24 more rows — Meera, Kiran, Divya, Arjun, Nisha, Vikram each x 4)

How to spot the accident: if a join returns a suspiciously huge number of rows, the ON clause is missing or wrong — that is a CROSS JOIN in disguise.

Q: SELF JOIN — a table joined to itself (employee and manager)

What it is, in plain words: the manager is also an employee, so the employees table points at ITSELF through manager_id. To put the worker and the boss on the SAME row, you join employees to employees with two aliases — w for the worker, m for the manager.

Syntax:

```
SELECT w.column, m.column
FROM   table w
JOIN   table m ON w.manager_id = m.emp_id;
```

Query:

```
SELECT w.emp_name AS employee, m.emp_name AS manager
FROM   employees w
LEFT JOIN employees m ON w.manager_id = m.emp_id
ORDER BY w.emp_name;
```

Output — 8 rows. The four top bosses have no manager, so their manager is NULL:

employee	manager
Arjun	Kiran
Asha	NULL
Divya	Kiran
Kiran	NULL
Meera	Asha
Nisha	NULL
Ravi	Asha
Vikram	NULL

Why LEFT here: an INNER self join would silently DROP the four bosses (they have no manager to match) — LEFT keeps everyone.

Q: Joining THREE tables — the pattern

What it is, in plain words: chain the joins left to right; each ON links the newest table to something already in the chain. Here we add locations, so every employee also gets their city.

Query:

```
SELECT e.emp_name, d.dept_name, l.city
FROM   employees e
JOIN   departments d ON e.dept_id    = d.dept_id
JOIN   locations   l ON d.location_id = l.location_id;
```

Output — 7 rows (these are inner joins, so Vikram and Legal are out):

emp_name	dept_name	city
Asha	Engineering	Bangalore
Ravi	Engineering	Bangalore
Meera	Engineering	Bangalore
Kiran	Sales	Mumbai
Divya	Sales	Mumbai
Arjun	Sales	Mumbai
Nisha	HR	Delhi

The joins in one line each (the memory sheet)

MEMORY HOOK: *"Inner = only the overlap. Left = left survives. Right = right survives. Full = both survive. Cross = everyone meets everyone. Self = the table looks in the mirror."*

SAY THIS: "A join matches rows on a shared key. INNER returns only the matches — unmatched rows on either side vanish. LEFT keeps every left row and fills the right with NULL where there is no match; RIGHT is the mirror image. FULL keeps both sides — the matches, plus the left-only rows, plus the right-only rows — and in MySQL I write it as a LEFT UNION RIGHT. CROSS is the cartesian product, which is also what a forgotten ON clause silently produces. And a SELF join joins the table to itself through aliases — that is how I show each employee beside their manager."

SECTION 6 — AGGREGATIONS: GROUP BY, HAVING, AND THE AGGREGATE FUNCTIONS

Q: What do the aggregate functions do?

Function	Returns	NULL behaviour
`COUNT(*)`	number of ROWS	counts everything, including NULLs
`COUNT(col)`	number of non-NULL values	SKIPS nulls
`COUNT(DISTINCT col)`	number of unique values	skips nulls
`SUM(col)`	total	skips nulls
`AVG(col)`	mean of non-null values	skips nulls
`MIN(col)` / `MAX(col)`	smallest / largest	skips nulls

```
SELECT COUNT(*) AS rows_all,               -- 8
       COUNT(dept_id) AS rows_with_dept,   -- 7 (Vikram has no dept)
       COUNT(DISTINCT dept_id) AS depts,    -- 3
       AVG(salary) AS avg_sal,              -- 65000.00
       MIN(salary) AS lowest,               -- 45000
       MAX(salary) AS highest               -- 90000
FROM employees;
```

Q: GROUP BY — collapsing rows into groups

Understand it: GROUP BY takes the rows, sorts them into buckets by the grouped column, and lets each aggregate run PER BUCKET instead of over the whole table. "One output row per group" is the mental model.

```
SELECT d.dept_name, COUNT(*) AS headcount, AVG(e.salary) AS avg_salary
FROM   employees e
JOIN   departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY headcount DESC;
```

OUTPUT:

dept_name	headcount	avg_salary
Engineering	3	76666.67
Sales	3	63333.33
HR	1	45000.00

The golden rule of GROUP BY (they WILL test it): every column in SELECT must either be inside an aggregate OR be listed in GROUP BY. And note what the aggregate counts: `COUNT(*)` counts the ROWS in each group (the `*` means any row, NULLs included) — the grouped column is only what defines the buckets, not what gets counted. `COUNT(col)` would count just the non-NULL values of that column inside each group. `SELECT dept_id, emp_name, COUNT(*)` is invalid — `emp_name` is not grouped and not aggregated, so which employee would the row show?

Q: HAVING — filtering the GROUPS

Understand it: WHERE filters ROWS before grouping; HAVING filters GROUPS after. If your condition mentions an aggregate (`COUNT(*) > 1`), it can only go in HAVING.

```
SELECT d.dept_name, COUNT(*) AS headcount
FROM   employees e
JOIN   departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
HAVING COUNT(*) > 1;
```

OUTPUT:

dept_name	headcount
Engineering	3
Sales	3

Q: WHERE + GROUP BY + HAVING in one query — the canonical shape

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM   employees e
JOIN   departments d ON e.dept_id = d.dept_id
WHERE  e.salary > 45000            -- 1. filter ROWS first (drops Nisha)
GROUP BY d.dept_name               -- 2. group the survivors
HAVING AVG(e.salary) > 60000       -- 3. filter the GROUPS
ORDER BY avg_salary DESC;          -- 4. sort the result
```

OUTPUT (HR vanishes — Nisha's 45000 was removed by WHERE, leaving HR with no rows):

dept_name	avg_salary
Engineering	76666.67
Sales	63333.33

MEMORY HOOK: *"WHERE cuts rows, HAVING cuts groups. Aggregates live in HAVING; row conditions live in WHERE."*

SAY THIS: "Aggregate functions collapse many rows into one value. GROUP BY buckets the rows so each aggregate runs per bucket — and the rule is every selected column must be aggregated or grouped. HAVING then filters those groups, which is the difference from WHERE: WHERE filters individual rows before grouping, HAVING filters the groups after. If my condition contains an aggregate like COUNT greater than one, it can only be in HAVING."

Q: Counting with zero — "show all departments including empty ones" (the LEFT JOIN + COUNT question)

Understand it: a plain INNER JOIN silently DROPS departments with no employees — Legal never appears, so you never notice the bug. To show all departments WITH their counts including zero, you must LEFT JOIN from departments and count the employee side.

```
SELECT d.dept_name, COUNT(e.emp_id) AS headcount
FROM   departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

OUTPUT (Legal shows 0):

dept_name	headcount
Engineering	3
Sales	3
HR	1
Legal	0

The subtle trap they probe: `COUNT(*)` here would return **1** for Legal (it counts the one NULL-filled row), while `COUNT(e.emp_id)` correctly returns **0** — because COUNT(column) skips NULLs. This is the single most common aggregation bug in interviews.

MEMORY HOOK: *"LEFT JOIN to keep the empties; COUNT(column) to count real ones, not the NULL row."*

SECTION 7 — SUBQUERIES

Q: What is a subquery, and where can it sit?

Understand it: a subquery is a query inside another query — a question whose answer feeds the outer question. It can sit in three places, and knowing the three is the interview answer.

Position	Example use	Returns
WHERE	find employees earning above the average	single value or a list
SELECT	show each employee's salary and the company max beside it	single value per row
FROM	treat a result as a temporary table (derived table)	a table

Q: Scalar subquery — a single value

```
SELECT emp_name, salary
FROM   employees
WHERE  salary > (SELECT AVG(salary) FROM employees);   -- 65000
```

OUTPUT (the four above 65000):

emp_name	salary
Asha	90000
Ravi	70000
Meera	70000
Divya	80000

The rule: a subquery in `=`, `>`, `<` must return ONE value — if it returns a column of many values, use `IN`/`ANY`/`ALL` instead.

Q: Multi-row subquery — IN, ANY, ALL

```
-- IN: employees in departments located in Mumbai or Delhi
SELECT emp_name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location IN ('Mumbai','Delhi'));

-- ALL: employees earning more than EVERY Sales salary
SELECT emp_name FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 2);

-- ANY: earning more than AT LEAST ONE Sales salary
SELECT emp_name FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 2);
```

The comparison to say out loud: `> ALL (list)` means "greater than the maximum"; `> ANY (list)` means "greater than the minimum" — that's the fastest way to remember them.

Q: Correlated subquery — the inner query references the outer row

Understand it: a normal subquery runs ONCE. A CORRELATED subquery re-runs for EVERY row of the outer query, because it references a column from the outer row. Powerful, but O(rows) executions — mention that trade-off and you sound senior.

```
-- each employee whose salary beats their OWN department's average
SELECT e.emp_name, e.dept_id, e.salary
FROM   employees e
WHERE  e.salary > (SELECT AVG(salary)
                   FROM   employees
```

```
                WHERE  dept_id = e.dept_id);   -- <-- references the outer row
```

OUTPUT:

emp_name	dept_id	salary
Asha	1	90000
Divya	2	80000

Q: EXISTS vs IN — the performance question

Understand it: `IN` builds the list of values and checks membership; `EXISTS` asks "does at least one matching row exist?" and can stop at the first hit. Both can return the same rows — the difference is how they behave and how they handle NULLs.

```
-- IN
SELECT dept_name FROM departments d
WHERE dept_id IN (SELECT dept_id FROM employees);

-- EXISTS (correlated — stops at the first match)
SELECT dept_name FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

	IN	EXISTS
Evaluates	the whole inner list first	per outer row, stops at first match
Best when	inner list is SMALL	inner table is LARGE / indexed
NULL danger	a NULL in the list can silently kill matches	immune — it checks existence, not equality
Readability	cleaner for short lists	clearer intent for "has related rows"

MEMORY HOOK: *"IN checks membership in a list; EXISTS checks existence. Small list -> IN; big table -> EXISTS."*

SAY THIS: "A subquery is a query inside a query — in the WHERE, SELECT, or FROM clause. A scalar subquery returns one value; IN, ANY and ALL handle lists — ALL means greater than the max, ANY means greater than the min. A correlated subquery re-runs per outer row, which is powerful but costs one execution per row. And for existence checks I prefer EXISTS over IN when the inner table is large, because EXISTS stops at the first match and isn't affected by NULLs."

SECTION 8 — WINDOW FUNCTIONS (explained from zero, step by step)

Q: What is a window function? (start with this one idea)

The one idea, in plain words: a normal aggregate with GROUP BY SQUASHES many rows into ONE row. A window function does the SAME calculation but KEEPS every row — it simply adds a new column beside each row showing that calculation.

Analogy — the class marks sheet. You have a marksheet with every student's marks. GROUP BY gives you a single line: "class average = 72". A window function gives you every student's line, each with "class average = 72" written beside it. The number is the same — but now every student can be compared to it, and no student's row is lost.

See the difference on real data. This table (7 employees) is the input for every example below.

emp_name	dept_id	salary
Asha	1	90000
Ravi	1	70000
Meera	1	70000
Kiran	2	60000
Divya	2	80000
Arjun	2	50000
Nisha	3	45000

If we use GROUP BY — the rows COLLAPSE to one per department (3 rows):

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM   employees
GROUP BY dept_id;
```

dept_id	avg_salary
1	76666.67
2	63333.33
3	45000.00

If we use a WINDOW function — every employee row is KEPT (7 rows), and each row carries its department's average:

```
SELECT emp_name, dept_id, salary,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM   employees;
```

emp_name	dept_id	salary	dept_avg
Asha	1	90000	76666.67
Ravi	1	70000	76666.67
Meera	1	70000	76666.67
Kiran	2	60000	63333.33
Divya	2	80000	63333.33
Arjun	2	50000	63333.33
Nisha	3	45000	45000.00

That is the whole idea: same calculation, but the rows survive. Now you can see that Asha earns more than her department's average — which the collapsed version could never show.

Q: The syntax — OVER, PARTITION BY, ORDER BY (the three words)

Syntax:

```
function() OVER (
    PARTITION BY column      -- optional: run the calculation separately per group
    ORDER BY column          -- optional: the order the calculation walks the rows in
)
```

What each word means:

- **OVER(...)** — "this function works across a SET of rows, not one row." That set of rows is called the window. Without OVER(), it is a normal aggregate.
- **PARTITION BY dept_id** — split the rows into groups and run the calculation separately INSIDE each group (like GROUP BY, but without collapsing the rows).
- **ORDER BY salary DESC** (inside OVER) — the order in which the calculation moves through the rows. Required for running totals, LAG/LEAD and ranking.

The one-line rule to remember: no `OVER()` = a normal aggregate (collapses rows). With `OVER()` = a window function (keeps rows).

Q: ROW_NUMBER, RANK, DENSE_RANK — numbering the rows

What they do, in plain words: all three put a number on every row, and the number is decided by the ORDER BY inside OVER — here, salary from highest to lowest. The ONLY thing that separates the three is how they treat a TIE (two rows with the same salary).

Function	On a tie, it gives	Sequence
ROW_NUMBER	different numbers (it breaks the tie)	1, 2, 3, 4
RANK	the same number, then it SKIPS	1, 2, 2, 4
DENSE_RANK	the same number, no skip	1, 2, 2, 3

Syntax:

```
ROW_NUMBER() OVER (ORDER BY col DESC)
RANK()       OVER (ORDER BY col DESC)
DENSE_RANK() OVER (ORDER BY col DESC)
```

Table (input) — a small team table with a tie in the middle. We use this tiny table on purpose: the tie is followed by one MORE row (Priya), which is exactly what makes the difference visible. Without a row after the tie, RANK and DENSE_RANK look identical.

name	salary
Asha	90000
Ravi	70000
Meera	70000
Priya	60000

Query:

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK()       OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense
```

```
FROM   team;
```

Output — now the three columns really differ:

name	salary	row_num	rnk	dense
Asha	90000	1	1	1
Ravi	70000	2	2	2
Meera	70000	3	2	2
Priya	60000	4	4	3

Read it row by row — this is the whole idea:

- **Asha (90000)** is the highest, so all three give her **1**.
- **Ravi and Meera (both 70000)** TIE. row_num must stay unique, so it gives them **2 and 3**. rank and dense_rank both give them **2**, because they share the same position.
- **Priya (60000)** is next. Because two people already shared position 2, **rank SKIPS 3 and gives Priya 4**, while **dense_rank carries straight on and gives Priya 3**.

Priya's row is the entire difference: rank runs 1, 2, 2, **4**; dense_rank runs 1, 2, 2, **3**. That is the only thing that ever separates them.

Where PARTITION BY fits — and it is a SEPARATE thing from ORDER BY. The two do different jobs:

- **ORDER BY** (inside OVER) = the BASIS — "rank the rows by this column."
- **PARTITION BY** = "start the count over again for each group."

The team example above used ORDER BY ALONE (no PARTITION BY). Here is the same idea on the employees table, with ROW_NUMBER only, so the difference is easy to read.

Without PARTITION BY — ONE running count across the whole table:

```
SELECT emp_name, dept_id, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM   employees;
```

emp_name	dept_id	salary	row_num
Asha	1	90000	1
Divya	2	80000	2
Ravi	1	70000	3
Meera	1	70000	4
Kiran	2	60000	5
Vikram	NULL	55000	6
Arjun	2	50000	7
Nisha	3	45000	8

With PARTITION BY dept_id — the count RESTARTS at 1 for each department:

```
SELECT emp_name, dept_id, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num
FROM   employees;
```

emp_name	dept_id	salary	row_num
Asha	1	90000	1
Ravi	1	70000	2
Meera	1	70000	3
Divya	2	80000	1
Kiran	2	60000	2
Arjun	2	50000	3
Nisha	3	45000	1
Vikram	NULL	55000	1

The one-line meaning: ORDER BY = "rank them by this"; PARTITION BY = "start over for each group." You can use ORDER BY alone, or both together — and the rank-vs-dense difference still shows inside any group that has a tie followed by another row.

MEMORY HOOK: *"The basis is the ORDER BY column. On a tie: ROW_NUMBER breaks it, RANK shares then skips, DENSE_RANK shares and does not skip."*

Q: LAG and LEAD — look at the previous or next row

What they do, in plain words: LAG gives you the value from the row ABOVE (the previous row), and LEAD from the row BELOW (the next row), in the ORDER BY order. This is how you compare a row with its neighbour — for example "this month's sales vs last month's".

Syntax:

```
LAG(col)  OVER (ORDER BY col)     -- value from the previous row
LEAD(col) OVER (ORDER BY col)     -- value from the next row
```

Query:

```
SELECT emp_name, hire_date, salary,
       LAG(salary)  OVER (ORDER BY hire_date) AS prev_salary,
       LEAD(salary) OVER (ORDER BY hire_date) AS next_salary
FROM   employees;
```

Output — the first row has no previous (NULL), the last row has no next (NULL):

emp_name	hire_date	salary	prev_salary	next_salary
Kiran	2019-11-20	60000	NULL	90000
Asha	2020-01-15	90000	60000	70000
Ravi	2021-03-10	70000	90000	45000
Nisha	2021-07-07	45000	70000	70000
Meera	2022-06-01	70000	45000	80000
Divya	2023-02-14	80000	70000	50000
Arjun	2024-05-05	50000	80000	NULL

Q: Running total — SUM with ORDER BY (and no PARTITION)

What it does, in plain words: with an ORDER BY inside OVER and NO PARTITION BY, the SUM adds up from the first row down to the current row — a running total. Each row shows the total SO FAR.

Query:

```
SELECT hire_date, emp_name, salary,
       SUM(salary) OVER (ORDER BY hire_date) AS running_total
FROM   employees
ORDER BY hire_date;
```

Output — each row adds the current salary to everything above it:

hire_date	emp_name	salary	running_total
2019-11-20	Kiran	60000	60000
2020-01-15	Asha	90000	150000
2021-03-10	Ravi	70000	220000
2021-07-07	Nisha	45000	265000
2022-06-01	Meera	70000	335000
2023-02-14	Divya	80000	415000
2024-05-05	Arjun	50000	465000

Q: Top 2 earners per department (the real-world use — window inside a subquery)

What it does, in plain words: you CANNOT filter on ROW_NUMBER in the same query, because the window column is created at SELECT time — after WHERE has already run. So you compute the number in a subquery, then filter OUTSIDE it.

Query:

```
SELECT dept_id, emp_name, salary
FROM (
    SELECT dept_id, emp_name, salary,
           ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
    FROM   employees
) t
WHERE rn <= 2;
```

Output — the top 2 by salary inside each department:

dept_id	emp_name	salary
1	Asha	90000
1	Ravi	70000
2	Divya	80000
2	Kiran	60000
3	Nisha	45000

Q: The two rules beginners always trip on

Rule 1 — you cannot use a window function in WHERE. `WHERE ROW_NUMBER() OVER (...) <= 2` is invalid. The window is computed after WHERE, so put it in a subquery (or a CTE) and filter outside, exactly as the example above does.

Rule 2 — window vs GROUP BY, in one line. GROUP BY gives you ONE row per group; a window function gives you EVERY row, plus the group's value beside it. If you need every row, you need a window — not GROUP BY.

MEMORY HOOK: *"GROUP BY collapses, a window keeps. OVER = across rows, PARTITION BY = restart per group, ORDER BY = the direction it walks. ROW_NUMBER unique, RANK skips, DENSE_RANK doesn't."*

SAY THIS: "A window function does a calculation across a set of rows but keeps every row — GROUP BY would collapse them. I write it with OVER, using PARTITION BY to restart the calculation per group and ORDER BY for the direction it moves. ROW_NUMBER gives unique numbers, RANK shares a number on ties then skips, DENSE_RANK shares without skipping. LAG and LEAD look at the previous and next rows, and SUM with ORDER BY gives a running total. One trap: a window function can't be used in WHERE, because it is computed at SELECT time — so I compute it in a subquery and filter outside."

SECTION 9 — CTEs (WITH clauses)

Q: What is a CTE, and why use it instead of a subquery?

Understand it: a CTE (Common Table Expression) is a NAMED temporary result you define with `WITH`, then use like a table. It reads top-to-bottom like a recipe, instead of inside-out like a nested subquery — and it can be referenced MULTIPLE times in the same query.

```
WITH dept_stats AS (
    SELECT dept_id, AVG(salary) AS avg_sal, COUNT(*) AS headcount
    FROM   employees
    GROUP BY dept_id
)
SELECT d.dept_name, s.avg_sal, s.headcount
FROM   dept_stats s
JOIN   departments d ON d.dept_id = s.dept_id
ORDER BY s.avg_sal DESC;
```

OUTPUT:

dept_name	avg_sal	headcount
Engineering	76666.67	3
Sales	63333.33	3
HR	45000.00	1

Why interviewers like CTEs: they make complex queries READABLE and let you reuse a result without repeating it. The trade-off line: "a CTE is not always materialised — some engines inline it like a subquery, so it's mainly a readability win, not automatically a performance one."

Q: Multiple CTEs in one query

```
WITH eng AS (SELECT * FROM employees WHERE dept_id = 1),
     sales AS (SELECT * FROM employees WHERE dept_id = 2)
SELECT 'Engineering' AS team, COUNT(*) AS n FROM eng
UNION ALL
SELECT 'Sales', COUNT(*) FROM sales;
```

Q: Recursive CTE — the org hierarchy (the "show the chain of command" question)

Understand it: a recursive CTE references ITSELF: start with the top bosses (the anchor), then repeatedly attach the next level down until no rows remain. This is the standard answer to "show the full reporting chain".

```
WITH RECURSIVE chain AS (
    SELECT emp_id, emp_name, manager_id, 1 AS level       -- anchor: the top bosses
    FROM   employees
    WHERE  manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.emp_name, e.manager_id, c.level + 1  -- next level
    FROM   employees e
    JOIN   chain c ON e.manager_id = c.emp_id
)
SELECT level, emp_name FROM chain ORDER BY level, emp_name;
```

OUTPUT:

level	emp_name
1	Asha
1	Kiran
1	Nisha
1	Vikram
2	Arjun
2	Divya
2	Meera
2	Ravi

MEMORY HOOK: *"A CTE is a named step in the recipe. Recursive = anchor + keep attaching the next level until nothing new appears."*

SECTION 10 — THE CLASSIC DIFFERENCES (the section they open every SQL interview with)

This whole section exists because these pairs are asked as direct "what's the difference between X and Y" questions. Each one: the answer, the why, and the sentence that makes it land.

Q: WHERE vs HAVING

	WHERE	HAVING
Filters	individual ROWS	GROUPS (after grouping)
Runs	BEFORE GROUP BY	AFTER GROUP BY
Aggregates allowed?	NO	YES
Can stand alone?	yes	no — needs GROUP BY (or an implicit single group)

```
SELECT dept_id, AVG(salary)
FROM   employees
WHERE  salary > 45000          -- rows first
GROUP BY dept_id
HAVING AVG(salary) > 60000;    -- then groups
```

SAY THIS: "WHERE filters rows before grouping and can't contain an aggregate; HAVING filters groups after grouping and is where aggregate conditions live."

Q: GROUP BY vs ORDER BY

	GROUP BY	ORDER BY
Purpose	COLLAPSE rows into groups	SORT the result
Changes row count?	YES (one row per group)	NO (same rows, reordered)
Position in flow	before SELECT output	last, after everything
Aggregates	required partner	irrelevant

The clarity line: "GROUP BY decides WHICH rows you get; ORDER BY decides WHAT ORDER they arrive in. You can use either without the other."

Q: DELETE vs TRUNCATE vs DROP

	DELETE	TRUNCATE	DROP
Removes	some/all ROWS	ALL rows	the whole TABLE
WHERE?	yes	no	no
Family	DML	DDL	DDL
Rollback	yes	usually no	no
Speed	slow	fast	instant
Identity reset	no	yes	n/a

SAY THIS: "DELETE removes rows and can be filtered and rolled back; TRUNCATE empties the whole table fast and usually can't be rolled back; DROP removes the table itself. Delete takes rows, truncate empties, drop deletes the table."

Q: UNION vs UNION ALL

	UNION	UNION ALL
Duplicates	removed	kept
Speed	slower (must dedupe)	faster (just stacks)
Use when	you need distinct rows	you know there are no dupes / want them

```
SELECT dept_id FROM employees
UNION
SELECT dept_id FROM departments;       -- duplicates removed

SELECT dept_id FROM employees
UNION ALL
SELECT dept_id FROM departments;       -- everything stacked
```

The rule for UNION to work: both queries must return the SAME number of columns with compatible types, in the same order — the column names come from the FIRST query.

Q: IN vs EXISTS

	IN	EXISTS
Question	"is this value in the list?"	"does a matching row exist?"
Evaluation	inner query runs once	per outer row, stops at first match
Best for	small inner lists	large tables / indexed joins
NULL safe?	no — a NULL in the list breaks matches	yes

Q: Primary key vs Unique key

	PRIMARY KEY	UNIQUE KEY
Uniqueness	yes	yes
NULLs	never — it's NOT NULL by definition	allows one NULL
Count per table	exactly ONE	many
Purpose	identifies the row	just prevents duplicates

Q: Clustered vs Non-clustered index

	Clustered	Non-clustered
What it is	table data stored in index order	separate structure -> row pointers
Per table	one	many
Access	direct — the leaf IS the row	extra hop to the row
Typical	primary key	indexes on search columns

Q: CHAR vs VARCHAR

	CHAR(n)	VARCHAR(n)
Storage	fixed — always n chars, padded	variable — only what's used
Speed	slightly faster (fixed)	slightly slower (length handling)
Use for	fixed codes (country 'IN', 'US')	names, emails, anything variable

Q: COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col)

```
SELECT COUNT(*)                AS all_rows,      -- 8: every row
       COUNT(dept_id)          AS with_dept,     -- 7: skips NULLs
       COUNT(DISTINCT dept_id) AS unique_depts   -- 3: unique non-null values
FROM employees;
```

The trap: `COUNT(*)` counts rows including NULL-filled ones; `COUNT(col)` skips NULLs. This is why a LEFT JOIN with COUNT(*) shows 1 for an empty group, not 0.

Q: RANK vs DENSE_RANK vs ROW_NUMBER

Function	Ties	Sequence example
ROW_NUMBER	broken arbitrarily	1, 2, 3, 4
RANK	share, then skip	1, 2, 2, 4
DENSE_RANK	share, no skip	1, 2, 2, 3

Q: DDL vs DML

	DDL	DML
Changes	structure	data
Commands	CREATE, ALTER, DROP, TRUNCATE	INSERT, UPDATE, DELETE
Rollback	generally not	yes
Auto-commit	usually yes	no (until COMMIT)

Q: Normalization vs Denormalization

	Normalization	Denormalization
Goal	remove redundancy, avoid anomalies	improve READ speed
Tables	more, smaller	fewer, wider (joined data repeated)
Writes	fast/safe	risk of inconsistency
Use	transactional (OLTP)	reporting/analytics (OLAP)

Q: View vs Table

	Table	View
Stores data	yes, physically	no — a saved QUERY
Fresh data	as written	always reflects current tables
Can insert/update	yes	only in simple cases

Q: Stored Procedure vs Function

	Stored Procedure	Function
Returns	zero or many values / result sets	exactly ONE value
Called from	a call statement	inside a SELECT/expression
Can do DML	yes	usually no (read-oriented)
Parameters	in, out, inout	typically in

Q: TRUNCATE and rollback — the gotcha

In most engines TRUNCATE is DDL and auto-commits, so it CANNOT be rolled back — unlike DELETE. State it plainly and you pre-empt the follow-up.

Q: AND vs OR precedence

`AND` binds tighter than `OR`. `WHERE dept_id = 1 OR dept_id = 2 AND salary > 60000` is really `dept_id = 1 OR (dept_id = 2 AND salary > 60000)`. Always bracket: `WHERE (dept_id = 1 OR dept_id = 2) AND salary > 60000`.

Q: NULL vs empty string

NULL = unknown / no value. `''` = a real, known, zero-length string. They are different: `'' = ''` is TRUE, `NULL = NULL` is not. `COUNT(col)` counts `''` but skips NULL.

Q: HAVING without GROUP BY

Legal — the whole result is treated as ONE group. `SELECT COUNT(*) FROM employees HAVING COUNT(*) > 5;` returns one row if the table has more than 5 rows, nothing otherwise.

SECTION 11 — TRANSACTIONS & CONCURRENCY

Q: What is a transaction, and what is ACID?

Understand it: a transaction is a group of statements that must succeed or fail AS ONE — the bank transfer classic: debit one account, credit the other; if the credit fails, the debit must vanish too. ACID is the four guarantees.

Letter	Property	Plain meaning
A	Atomicity	all-or-nothing — every statement or none
C	Consistency	data moves from one valid state to another
I	Isolation	concurrent transactions don't corrupt each other
D	Durability	once committed, it survives a crash

MEMORY HOOK: *"All-or-nothing, Consistent rules, Isolated from each other, Durable on disk."*

Q: The TCL commands

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 5000 WHERE acc_id = 1;
UPDATE accounts SET balance = balance + 5000 WHERE acc_id = 2;
COMMIT;                -- make both permanent

-- or, if something went wrong:
ROLLBACK;              -- undo everything since START TRANSACTION

-- partial undo — roll back to a marked point:
SAVEPOINT before_credit;
UPDATE accounts SET balance = balance + 5000 WHERE acc_id = 2;
ROLLBACK TO before_credit;   -- undo only the credit
COMMIT;
```

Q: What are the isolation levels? (the levels table)

Level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	no	possible	possible
REPEATABLE READ	no	no	possible
SERIALIZABLE	no	no	no

The one-liner: higher isolation = more correctness, less concurrency. The three anomalies: a dirty read sees uncommitted data; a non-repeatable read sees a row change between two reads; a phantom read sees NEW rows appear in a repeated range query.

Q: What is a deadlock?

Two transactions each hold a lock the other needs, so both wait forever — the engine detects it and kills one. Prevention: touch tables in a consistent order, keep transactions short, and use the right isolation level.

SAY THIS: "A transaction groups statements into one atomic unit — ACID says it's all-or-nothing, keeps data consistent, isolates concurrent work, and survives crashes once committed. I control it with COMMIT, ROLLBACK and SAVEPOINT. Isolation levels trade correctness against concurrency, and a deadlock is two transactions waiting on each other's locks, which the engine resolves by aborting one."

SECTION 12 — RDBMS THEORY QUESTIONS

Q: What is normalization? Explain 1NF, 2NF, 3NF with a tiny example.

Understand it: normalization is tidying a table so each fact lives in exactly ONE place — removing redundancy that causes update/insert/delete anomalies. Each normal form fixes one kind of mess.

Form	Rule	The mess it removes
1NF	atomic values, no repeating groups	a `phones` column holding "9999,8888"
2NF	1NF + no partial dependency on part of a composite key	a non-key column depending on only half the key
3NF	2NF + no transitive dependency	`dept_name` stored in employees (it depends on dept_id, not emp_id)
BCNF	stricter 3NF — every determinant is a candidate key	rare edge cases

The example to tell: storing `dept_name` inside `employees` breaks 3NF — the name really belongs to `departments`. Split it out and you fix it: that is exactly why our sample data has two tables.

MEMORY HOOK: *"1NF atomic, 2NF whole key, 3NF nothing but the key."*

Q: What is denormalization and when would you use it?

Deliberately re-introducing redundancy to make READS faster — e.g. copying `dept_name` into a reporting table so dashboards don't join. Trade-off: faster reads, riskier writes. Use it for analytics/OLAP, not for transactional systems.

Q: What is an entity-relationship model, and what are the relationship types?

An ER model diagrams entities (tables), attributes (columns) and relationships. Relationship cardinalities: **1:1**, **1:many** (one department has many employees — the common one), and **many:many** (students<->courses, needs a junction table).

Q: What is referential integrity?

The guarantee that a foreign key always points to a real row — you cannot insert an employee with a `dept_id` that doesn't exist, and you cannot delete a department that still has employees (unless the FK says ON DELETE CASCADE).

Q: What is a schema? What is an instance?

The **schema** is the design — tables, columns, types, constraints (rarely changes). The **instance** is the actual data in it at a moment in time (changes constantly).

Q: SQL vs NoSQL — when do you choose which?

	SQL (relational)	NoSQL
Schema	fixed, enforced	flexible / schemaless
Best for	relationships, transactions, integrity	scale, speed, unstructured data
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis
Consistency	strong (ACID)	often eventual (BASE)

The balanced answer: "SQL for structured, relational, transactional data where integrity matters; NoSQL when the shape varies or the write volume is huge and I can trade strict consistency for scale. Most real systems use both."

Q: What are the common integrity constraints?

Domain (type/CHECK), entity (PRIMARY KEY), referential (FOREIGN KEY), and business rules. They are the database's own guardrails so bad data never enters.

Q: What is a view, a stored procedure, a trigger, a cursor?

Object	What it is	Why it exists
View	a saved SELECT treated as a table	simplify complex queries, restrict columns
Stored procedure	pre-compiled SQL on the server	reuse logic, reduce round-trips
Trigger	code that fires automatically on INSERT/UPDATE/DELETE	enforce rules, audit changes
Cursor	a pointer to walk a result row by row	procedural logic over a set

Q: What is the difference between a database and a data warehouse?

A database (OLTP) serves day-to-day transactions, normalized and fast per-row. A warehouse (OLAP) serves analysis, denormalized/star-schema, fast per-scan. Same SQL, different purpose.

SAY THIS: "Normalization removes redundancy in steps — 1NF makes values atomic, 2NF removes partial dependencies on part of a key, 3NF removes transitive dependencies, so every fact lives in one place. Denormalization reverses it for read speed in analytics. An ER model maps entities and their relationships, referential integrity keeps foreign keys pointing at real rows, and the schema is the design while the instance is the data at a moment in time."

SECTION 13 — THE QUERY PROBLEM BANK (the ones they make you write on the spot)

All problems use the same employees/departments data from Section 5. Each one gives the task, the query, and the expected output — practise writing the query, then check your result against the output.

Q13.1 Second highest salary (the single most-asked query — know all three ways)

```
-- Way 1: subquery — max below the overall max
SELECT MAX(salary) AS second_highest
FROM   employees
WHERE  salary < (SELECT MAX(salary) FROM employees);
```

```
-- Way 2: LIMIT/OFFSET — sort desc, skip 1, take 1 (MySQL)
SELECT DISTINCT salary
FROM   employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

```
-- Way 3: window function — rank then pick rank 2 (works per group too)
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM   employees
) t
WHERE rnk = 2;
```

OUTPUT (all three): `80000`

Why three ways matter: Way 1 fails to give per-department answers, Way 2 needs DISTINCT for ties, Way 3 is the only one that generalises to "Nth per group" — say that and you've answered the follow-up before it's asked.

Q13.2 Nth highest salary (make it a parameter)

```
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM   employees
) t
WHERE rnk = 3;                 -- change 3 to N
```

OUTPUT (N=3): `70000`

Q13.3 Highest salary PER DEPARTMENT (the GROUP BY + join classic)

```
SELECT d.dept_name, MAX(e.salary) AS top_salary
FROM   employees e
JOIN   departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

OUTPUT:

dept_name	top_salary
Engineering	90000
Sales	80000
HR	45000

Q13.4 The employee(s) earning the top salary in EACH department (needs a window, not just MAX)

```
SELECT dept_id, emp_name, salary FROM (
    SELECT dept_id, emp_name, salary,
           RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM   employees
) t
WHERE rnk = 1;
```

OUTPUT:

dept_id	emp_name	salary
1	Asha	90000
2	Divya	80000
3	Nisha	45000

Why MAX alone is not enough: `MAX(salary)` gives the number, not WHO earned it — to get the name you need the row, so you rank and filter.

Q13.5 Find duplicate rows (and delete them keeping one)

```
-- find duplicates by name
SELECT emp_name, COUNT(*) AS copies
FROM   employees
GROUP BY emp_name
HAVING COUNT(*) > 1;

-- delete duplicates, keep the lowest emp_id
DELETE FROM employees
WHERE  emp_id NOT IN (SELECT MIN(emp_id) FROM employees GROUP BY emp_name);
```

OUTPUT of the find (on our data): no rows — all names are unique. **What to say:** "On this dataset there are no duplicates, so the query returns nothing — which is itself the correct result to report."

Q13.6 Employees earning more than their manager (self join — a favourite)

```
SELECT w.emp_name AS employee, w.salary AS emp_sal,
       m.emp_name AS manager,  m.salary AS mgr_sal
FROM   employees w
JOIN   employees m ON w.manager_id = m.emp_id
WHERE  w.salary > m.salary;
```

OUTPUT:

employee	emp_sal	manager	mgr_sal
Divya	80000	Kiran	60000

Q13.7 Departments with NO employees (the anti-join)

```
SELECT d.dept_name
FROM   departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE  e.emp_id IS NULL;
```

OUTPUT: `Legal`

The pattern to memorise: "LEFT JOIN then WHERE right-side IS NULL" is THE anti-join — it finds rows with no match. The `EXISTS`/`NOT EXISTS` version is the alternative.

Q13.8 Employees hired in the last 30 days (date filtering)

```
SELECT emp_name, hire_date
FROM   employees
WHERE  hire_date >= CURDATE() - INTERVAL 30 DAY;   -- MySQL
-- Oracle/PostgreSQL:  WHERE hire_date >= CURRENT_DATE - INTERVAL '30 days'
```

Q13.9 Running total of salaries in hire order (window)

```
SELECT hire_date, emp_name, salary,
       SUM(salary) OVER (ORDER BY hire_date) AS running_total
FROM   employees
ORDER BY hire_date;
```

OUTPUT (first rows):

hire_date	emp_name	salary	running_total
2019-11-20	Kiran	60000	60000
2020-01-15	Asha	90000	150000
2021-03-10	Ravi	70000	220000
2021-07-07	Nisha	45000	265000

Q13.10 Employees earning above the company average

```
SELECT emp_name, salary
FROM   employees
WHERE  salary > (SELECT AVG(salary) FROM employees);
```

OUTPUT (average is 65000): Asha 90000, Ravi 70000, Meera 70000, Divya 80000.

Q13.11 Employees earning above THEIR department's average (correlated)

```
SELECT e.emp_name, e.dept_id, e.salary
FROM   employees e
WHERE  e.salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);
```

OUTPUT: Asha (90000), Divya (80000).

Q13.12 Count employees per department INCLUDING zero (the COUNT trap)

```
SELECT d.dept_name, COUNT(e.emp_id) AS headcount
FROM   departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

OUTPUT: Engineering 3, Sales 3, HR 1, **Legal 0**.

Q13.13 Second-highest salary per department (window, per group)

```
SELECT dept_id, emp_name, salary FROM (
    SELECT dept_id, emp_name, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM   employees
) t
WHERE rnk = 2;
```

OUTPUT: dept 1 -> Ravi/Meera 70000; dept 2 -> Kiran 60000.

Q13.14 Find the manager's name for every employee (self join, LEFT to keep bosses)

```
SELECT w.emp_name AS employee, COALESCE(m.emp_name, 'No manager') AS manager
FROM   employees w
LEFT JOIN employees m ON w.manager_id = m.emp_id;
```

Q13.15 Rows where two columns have the same value (the "same value" question)

```
-- e.g. employees whose salary equals another employee's salary (tied)
SELECT a.emp_name, b.emp_name AS same_salary_as, a.salary
FROM   employees a
JOIN   employees b ON a.salary = b.salary AND a.emp_id < b.emp_id;
```

OUTPUT: Ravi <-> Meera (both 70000).

Q13.16 Salary bands with CASE (conditional column — very common)

```
SELECT emp_name, salary,
       CASE WHEN salary >= 80000 THEN 'High'
            WHEN salary >= 60000 THEN 'Medium'
            ELSE 'Low' END AS band
FROM   employees
ORDER BY salary DESC;
```

OUTPUT (first rows): Asha -> High; Divya -> High; Ravi/Meera -> Medium; Kiran -> Medium; Arjun -> Low; Nisha -> Low.

Q13.17 Count of employees per manager (including managers with none)

```
SELECT m.emp_name AS manager, COUNT(e.emp_id) AS reports
FROM   employees m
LEFT JOIN employees e ON e.manager_id = m.emp_id
GROUP BY m.emp_name
HAVING COUNT(e.emp_id) > 0;
```

OUTPUT: Asha 2 (Ravi, Meera), Kiran 2 (Divya, Arjun).

Q13.18 Employees with no manager (top of the chain)

```
SELECT emp_name FROM employees WHERE manager_id IS NULL;
```

OUTPUT: Asha, Kiran, Nisha, Vikram.

Q13.19 The highest-paid employee overall (name, not just number)

```
SELECT emp_name, salary FROM employees ORDER BY salary DESC LIMIT 1;
```

OUTPUT: Asha 90000.

Q13.20 Percentage share of each department in total salary

```
SELECT d.dept_name,
       SUM(e.salary) AS dept_total,
       ROUND(SUM(e.salary) * 100.0 / (SELECT SUM(salary) FROM employees), 2) AS pct
FROM   employees e
JOIN   departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

OUTPUT:

dept_name	dept_total	pct
Engineering	230000	44.23
Sales	190000	36.54
HR	45000	8.65

Q13.21 Pagination — page 2 of 3-per-page

```
SELECT * FROM employees ORDER BY emp_id LIMIT 3 OFFSET 3;   -- rows 4-6
```

Q13.22 Swap two values in a column (the tricky update)

```
UPDATE employees
SET salary = CASE emp_id WHEN 2 THEN 70000 WHEN 3 THEN 70000 END
WHERE emp_id IN (2,3);    -- for equal values this is a no-op; for real swaps use a temp
```

Q13.23 Employees whose name starts with a vowel

```
SELECT emp_name FROM employees
WHERE emp_name REGEXP '^[AEIOUaeiou]';   -- MySQL; use LIKE 'A%' OR 'E%'... portably
```

Q13.24 Find missing numbers in a sequence (the "gap" question)

```
SELECT (t1.emp_id + 1) AS missing_from
FROM   employees t1
LEFT JOIN employees t2 ON t1.emp_id + 1 = t2.emp_id
WHERE  t2.emp_id IS NULL;
```

Why it works: join each id to the "next expected" id; where the next doesn't exist, there's a gap.

Q13.25 Top 2 salaries per department (top-N per group — window)

```
SELECT dept_id, emp_name, salary FROM (
    SELECT dept_id, emp_name, salary,
           ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
    FROM   employees
) t
WHERE rn <= 2;
```

OUTPUT: dept 1 -> Asha 90000, Ravi 70000; dept 2 -> Divya 80000, Kiran 60000; dept 3 -> Nisha 45000.

MEMORY HOOK for the whole bank: *"Second-highest: subquery or window. Per-group top: rank in a subquery, filter outside. No-match rows: LEFT JOIN then IS NULL. Above-average: subquery in WHERE. Running total: SUM OVER ORDER BY."*

SECTION 14 — DATABASE TESTING FOR QA (because you are a tester, not a DBA)

Q: As a QA engineer, what do you actually test in a database?

Understand it: you verify that the application WROTE the right data, in the right shape, in the right place — and that nothing broke the rules. Four buckets, and interviewers expect all four.

1. **Data integrity** — does the value saved match what was entered? (order total = sum of line items)
2. **Referential integrity** — do foreign keys point at real rows? (no orphan order with a missing customer)
3. **Data correctness across the UI/DB** — UI shows Rs.500, DB column says 500.00?
4. **Constraint & business rules** — can a negative salary be saved? Should it be blocked?

```
-- the check you write most often: compare UI input to what landed
SELECT order_id, total_amount, status FROM orders WHERE order_id = 1024;

-- referential integrity sweep — orphan rows
SELECT o.order_id FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;      -- any row here is a DEFECT
```

Q: How do you validate an ETL / data migration?

Row counts match source to target; distinct keys are unique in the target; totals (SUM of amount) reconcile; no NULLs in mandatory columns; and a spot-check of a few transformed rows against the source.

```
SELECT COUNT(*) FROM source_orders;   -- compare with target
SELECT COUNT(*) FROM target_orders;

SELECT SUM(amount) FROM source_orders;
SELECT SUM(amount) FROM target_orders;   -- must reconcile
```

Q: The common DB-testing interview questions, one line each

Question	The answer
How do you connect to a DB from a test?	JDBC in Java, a driver + connection string; query and assert in the test
How do you compare UI and DB?	fetch the value from both, assert equality in code
What is a database deadlock in testing?	two sessions blocked — reproduce with concurrency, report as a defect
How do you test stored procedures?	call with known inputs, assert the output rows/values and side effects
How do you check performance?	EXPLAIN the query, check index usage, measure on realistic data volumes

```
// the JDBC shape QA interviewers expect you to sketch
Connection con = DriverManager.getConnection(url, user, pass);
Statement  st  = con.createStatement();
ResultSet  rs  = st.executeQuery("SELECT status FROM orders WHERE order_id=1024");
rs.next();
Assert.assertEquals(rs.getString("status"), "SHIPPED");
```

SAY THIS: "As a tester I use SQL to verify the application, not to design schemas. I check that what the UI sent is what the DB stored, that foreign keys never dangle, that totals reconcile, and that constraints actually block bad data. For migrations I compare row counts and sums between source and target, and for a specific defect I query the exact row the UI touched."

SECTION 15 — RAPID-FIRE, FULLY EXPLAINED (final revision)

Each line is the one-breath answer; the paragraph is what you say on "explain more".

- **DDL vs DML vs DQL?** DDL defines structure (CREATE/ALTER/DROP/TRUNCATE), DML changes data (INSERT/UPDATE/DELETE), DQL reads (SELECT). The trap is TRUNCATE — it looks like a delete but it's DDL, so it usually can't be rolled back.
- **Execution order?** FROM/JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT. It explains why SELECT aliases can't be used in WHERE (they don't exist yet) and why aggregates filter in HAVING, not WHERE.
- **WHERE vs HAVING?** WHERE filters rows before grouping; HAVING filters groups after. Aggregates only in HAVING.
- **GROUP BY vs ORDER BY?** GROUP BY collapses rows into groups (changes the count); ORDER BY just sorts the result (same rows, new order).
- **INNER vs LEFT join?** INNER keeps only matching rows from both sides; LEFT keeps every left row and NULLs the right where there's no match — that's the join for "including the ones with nothing".
- **Primary key vs unique?** Both stop duplicates; primary key is never NULL and there's exactly one per table, unique can hold one NULL and there can be many.
- **NULL handling?** NULL is unknown — `= NULL` never matches, so use IS NULL; aggregates skip NULLs, so COUNT(col) differs from COUNT(*).
- ***COUNT()* vs COUNT(col)?*** *Star counts rows (including NULL-filled ones); column counts non-NULL values. That's why LEFT JOIN + COUNT()* shows 1 for an empty group instead of 0.
- **DELETE vs TRUNCATE vs DROP?** Delete removes rows (rollback-able); truncate empties the whole table (fast, usually not rollback-able); drop removes the table.
- **UNION vs UNION ALL?** UNION removes duplicates (slower); UNION ALL keeps everything (faster). Both need the same column count and types.
- **RANK vs DENSE_RANK vs ROW_NUMBER?** Row_number is always unique; rank shares a number on ties then skips; dense_rank shares and doesn't skip.
- **Second highest salary?** MAX below the max, or ORDER BY DESC LIMIT 1 OFFSET 1, or DENSE_RANK = 2 in a subquery — the window version also gives per-group answers.
- **How to find rows with no match?** LEFT JOIN then WHERE the right-side key IS NULL — the anti-join.
- **Subquery vs JOIN?** A join combines rows side by side; a subquery answers a question that feeds the outer query. Joins are usually faster for retrieving columns; subqueries read cleaner for existence/aggregate checks.
- **Index — why and when?** It's a sorted structure that turns a full scan into a lookup; great for WHERE/JOIN/ORDER BY columns, but every index slows writes, so you don't index everything.
- **ACID?** Atomicity (all or nothing), Consistency (valid states), Isolation (concurrent work doesn't corrupt), Durability (survives a crash).
- **Normalization in one line?** Each fact in one place — 1NF atomic, 2NF whole key, 3NF nothing but the key.
- **What is a CTE?** A named temporary result defined with WITH — it makes a complex query read top-to-bottom and lets you reuse a result.
- **Correlated vs non-correlated subquery?** Non-correlated runs once; correlated references the outer row and re-runs per row (powerful, but O(rows)).
- **How do you debug a slow query?** Read the plan with EXPLAIN — check whether it used an index, how many rows it examined, and whether the join order is sensible.

SECTION 16 — CLOSING ANSWER + REVISION PLAN

Q: "How strong is your SQL for this role?"

> *"I'm comfortable with the whole everyday surface — filtering, joins of every kind, aggregations with GROUP BY and HAVING, subqueries, and window functions for things like ranking and running totals. I know the execution order cold, so I can explain WHY a query behaves the way it does, and I can read an EXPLAIN plan when something is slow. And because I come from a testing background, I also use SQL to verify the application — comparing what the UI saved against what the database actually holds, checking referential integrity, and reconciling data in migrations."*

MEMORY HOOK: *"From-Where-Group-Having-Select-Order-Limit — and every difference question is just two of those steps compared."*

Last-day revision plan

1. Say the execution order out loud 10 times — it answers more questions than any other single fact.
2. Write the second-highest-salary query in all three ways from memory.
3. Recite the differences table: WHERE/HAVING, GROUP/ORDER, DELETE/TRUNCATE/DROP, UNION/UNION ALL, RANK/DENSE_RANK/ROW_NUMBER.
4. Reproduce the join output table for our 8 employees — if you can predict INNER vs LEFT vs RIGHT results, joins are done.
5. Write one window query (top-N per group) and one anti-join (departments with no employees) from scratch.
6. Read the rapid-fire section twice the night before.

Walk in and treat the schema like it's your own — after this guide, it is.
